Dictionaries and Sets
Lec-22_23_24_25

Dictionaries

- A dictionary is an **unordered** collection which stores key–value pairs that map **immutable keys** to **values**, just as a conventional dictionary maps words to definitions.
- What data types can qualify for being a key?
  - Strings
  - Tuples (that do not contain mutable items)
  - ints, floats, chars
- A dictionary's keys must be immutable (such as strings, numbers or tuples) and unique (that is, no duplicates).

Dictionaries

- Syntax for creating dictionaries.
- To find number of keys : print(len(dictionary3))
- To print the entire dictionary: print(dictionary3)

```
# Creates an empty dictionary

dictionary1 = {}
dictionary2 = dict()

# Creates a dictionary with key-value pairs

dictionary3 = {'a':1,
               (1,2,3):2.5,
               'cat':[1,2,3],
               4:'hehehe'
              }
```

Dictionaries

- To iterate over the key-value pairs use .items() method with an iterator.
- The iterator is a tuple.
- The iterator can be unpacked by replacing the iterator with two variables.
- Dictionary items are not subscriptable. Example:
  - x = dictionary3.item()
  - print(x[0]) # will cause an error

```
for iterator in dictionary3.items():
    print(iterator)
```

✓ 0.0s

```
('a', 1)
((1, 2, 3), 2.5)
('cat', [1, 2, 3])
(4, 'hehehe')
```

```
for i, j in dictionary3.items():
    print(i, j)
```

✓ 0.0s

```
a 1
(1, 2, 3) 2.5
cat [1, 2, 3]
4 hehehe
```

Dictionaries

- To check if a dictionary is empty or not, the len() method or if else can be used.
- To empty a dictionary we can use the .clear() method

```
# Check if dict is empty or not
# Using len method
print(len(dictionary3))
# Using if else
if dictionary3:
    print('Not Empty')
else:
    print('Empty')
```

✓ 0.0s

```
4
Not Empty
```

```
dictionary3.clear()
if dictionary3:
    print("Not Empty")
else:
    print("Empty")
```

✓ 0.0s

```
Empty
```

Dictionaries

For this section, let's begin by creating and displaying the dictionary `roman_numerals`. We intentionally provide the incorrect value `100` for the key `'X'`, which we'll correct shortly:

```
In [1]: roman_numerals = {'I': 1, 'II': 2, 'III': 3, 'V': 5, 'X': 100}

In [2]: roman_numerals
Out[2]: {'I': 1, 'II': 2, 'III': 3, 'V': 5, 'X': 100}
```

Accessing the Value Associated with a Key

Let's get the value associated with the key `'V'`:

```
In [3]: roman_numerals['V']
Out[3]: 5
```

Updating the Value of an Existing Key–Value Pair

You can update a key's associated value in an assignment statement, which we do here to replace the incorrect value associated with the key `'X'`:

```
In [4]: roman_numerals['X'] = 10

In [5]: roman_numerals
Out[5]: {'I': 1, 'II': 2, 'III': 3, 'V': 5, 'X': 10}
```

Adding a New Key–Value Pair

Assigning a value to a nonexistent key inserts the key–value pair in the dictionary:

```
In [6]: roman_numerals['L'] = 50

In [7]: roman_numerals
Out[7]: {'I': 1, 'II': 2, 'III': 3, 'V': 5, 'X': 10, 'L': 50}
```

String keys are case sensitive. Assigning to a nonexistent key inserts a new key–value pair. This may be what you intend, or it could be a logic error.

Dictionaries

- You may insert and update key–value pairs using dictionary method update.
- Method update can convert keyword arguments into key–value pairs to insert.

```
# Method 1 to add or update a key-value pair
dictionary1['GTA Vice City'] = 'Tommy Vercetti'
# Method 2 to add or update a key-value pair
dictionary1.update({'GTA San Andreas':'Carl Johnson'})
# Method 3 to add or update a key-value pair
dictionary1.update(GTA_V='Trevor, Michael, Franklin')
# Method 4 to add multiple key-value pairs
# by passing a list of tuples with each tuple containing
# key value pair separated by (,)
dictionary1.update([
                    ('GTA Vice City','Tommy'),
                    ('GTA San Andreas','Carl')
                    ])
dictionary1
```

✓ 0.0s

```
{'GTA Vice City': 'Tommy',
 'GTA San Andreas': 'Carl',
 'GTA_V': 'Trevor, Michael, Franklin'}
```

Dictionaries

Removing a Key–Value Pair

You can delete a key–value pair from a dictionary with the `del` statement:

```
In [8]: del roman_numerals['III']

In [9]: roman_numerals
Out[9]: {'I': 1, 'II': 2, 'V': 5, 'X': 10, 'L': 50}
```

You also can remove a key–value pair with the dictionary method **pop**, which returns the value for the removed key:

```
In [10]: roman_numerals.pop('X')
Out[10]: 10

In [11]: roman_numerals
Out[11]: {'I': 1, 'II': 2, 'V': 5, 'L': 50}
```

Testing Whether a Dictionary Contains a Specified Key

Operators `in` and `not in` can determine whether a dictionary contains a specified key:

```
In [16]: 'V' in roman_numerals
Out[16]: True

In [17]: 'III' in roman_numerals
Out[17]: False

In [18]: 'III' not in roman_numerals
Out[18]: True
```

Attempting to Access a Nonexistent Key

Accessing a nonexistent key results in a `KeyError`:

```
In [12]: roman_numerals['III']
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
<ipython-input-12-ccd50c7f0c8b> in <module>()
----> 1 roman_numerals['III']

KeyError: 'III'
```

You can prevent this error by using dictionary method **get**, which normally returns its argument's corresponding value. If that key is not found, `get` returns `None`. IPython does not display anything when `None` is returned in snippet `[13]`. If you specify a second argument to `get`, it returns that value if the key is not found:

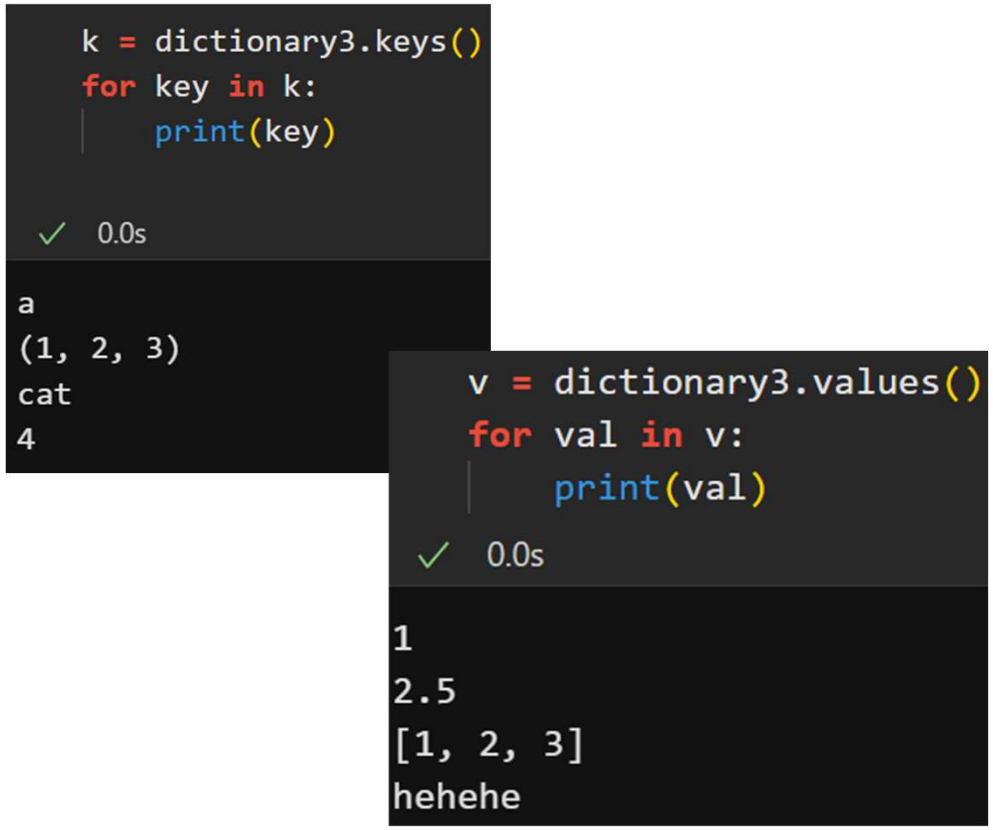
```
In [13]: roman_numerals.get('III')

In [14]: roman_numerals.get('III', 'III not in dictionary')
Out[14]: 'III not in dictionary'

In [15]: roman_numerals.get('V')
Out[15]: 5
```

Dictionaries

- To fetch all the key-value pairs we use the .items() method.
- To fetch all the keys only we can use the .keys() method.
- To fetch all the values only we can use the .values() method.



Dictionaries

- .items(), .keys(), .values() return a dict object otherwise called as a **view.**
- An iteration over the view displays the current contents of a dict.
- Views do not maintain their own copy of data.
- **Do not modify a dictionary while iterating through a view!** According to Section 4.10.1 of the Python Standard Library documentation,1 either you'll get a **RuntimeError or the loop might not process all of the view's values**.

```
dictionary4 = {'A':'apple', 'B': 'ball', 'C': 'cat'}
key_collection = dictionary4.keys() # view created once
for k in key_collection:
    print(k, end=' ') # Prints A B C
dictionary4['D'] = 'dog'
print()
for k in key_collection:
    print(k, end=' ')
# Prints A B C D without any updation to key_collection
```

✓ 0.0s

```
A B C
A B C D
```

Dictionaries

- A dict **view** can be converted into a list, tuple by calling the equivalent method and passing the view as an argument.
- Modifying this iterable doesn't modify the dictionary.
- To process items in sorted order, you can use built-in function **sorted.**

```
a, b, c = list(dictionary4.keys()), \
          list(dictionary4.values()), \
          list(dictionary4.items())
a[0], b[0], c[0] = 1, 2, 3
print(a, b, c, sep='\n')
print(dictionary4)
```

✓ 0.0s

```
[1, 'B', 'C', 'D']
[2, 'ball', 'cat', 'dog']
[3, ('B', 'ball'), ('C', 'cat'), ('D', 'dog')]
{'A': 'apple', 'B': 'ball', 'C': 'cat', 'D': 'dog'}
```

Dictionaries

- Dicts can be compared using the comparison operators **== and != only.**
- **<=, <, >, >=** are not allowed to compare dicts.
- An equals (==) comparison evaluates to True if both dictionaries have the same key–value pairs, regardless of the order in which those key–value pairs were added to each dictionary

Dictionary Comprehension

- Same as list comprehension except the expression is slightly altered.
- new_dict =

{**expression_for_key:expression_for_value**

for **member** in **iterable**

if **condition**

}

- Q : Write a DC for squaring odd numbers and cubing even numbers within [1-10]

```
''' Write a dictionary comprehension for
    generating squares of odd and
    cubes of even numbers from [1-10]
'''
def sq_cb(x):
    if x % 2:
        return x * x
    else:
        return x * x * x

ans = {i:sq_cb(i) for i in range(1,11)}
ans
```

DCs

- WADC that stores the word and its frequency.

Example string : ‘Apple Banana Apple Cheese Hot-Dog’

O/P : {Apple:2, Banana:1, Cheese:1, Hot-Dog:1}

DCs Answers

```
"""
WADC to store the word and
its count from a string
"""
s = '''
    Apple Banana Apple Cheese Hot-Dog
    '''

lst = s.split()
ans = {word:lst.count(word) for word in lst}
ans
```

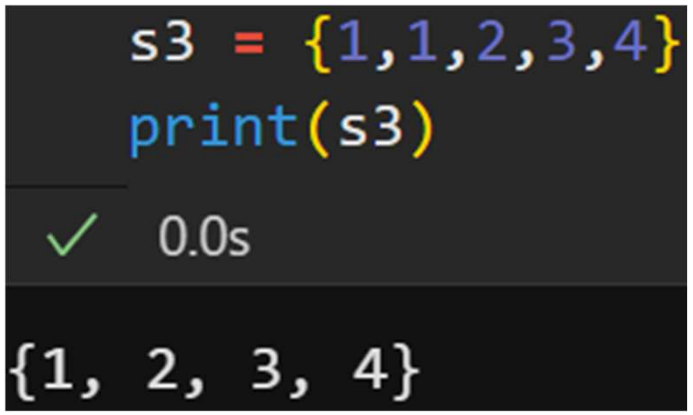
Sets

- A set is an unordered collection of unique values.
- Sets may contain only immutable objects, like strings, ints, floats and tuples that contain only immutable elements.
- Though sets are iterable, they are not sequences and do not support indexing and slicing with square brackets, [].
- Dictionaries also do not support slicing.

```
# To create an empty set
s1 = set()
# To create a set from an iterable
s2 = set([1,2,3,4])
# or
s3 = {1,2,3,4}
```

Sets

- Set will remove duplicate elements when adding elements from the iterable.
- Elements inside the set are unordered.
- Use len(set_var) to find the number of items in a set.
- To check if a value is in the set, the in operator can be used.



Sets

- To add elements into a set use the .add() method and pass an immutable object as an argument.
- To remove a specific element use the .remove() method and pass an immutable object as an argument. Trying to remove an element that doesn't exist using the .remove() method causes an error.
- Use the method .discard() to remove items safely.

```
s3 = {4,2,'abc',(1,2,3)}
s3.add('a')
s3.add(('Batman', 'Arkham', 'Asylum'))
print(s3)
```

```
s3.discard('b')
print(s3)
s3.discard('a')
print(s3)
```

✓ 0.0s

```
{2, 4, 'abc', (1, 2, 3), 'a', ('Batman', 'Arkham', 'Asylum')}
{2, 4, 'abc', (1, 2, 3), ('Batman', 'Arkham', 'Asylum')}
```

Sets

- A random element from the set can be removed using the .pop() method.
- In sets the .pop() method **doesn’t take any argument unlike dictionaries** or **lists**.
- Use the .clear() method to empty the set.

Sets

- Like lists you can check if an item is in the set, by using the in operator.
- Example : 1 in s1, ‘ABC’ in s2
- Sets are iterable so it can be iterated in for loop.
- **Frozenset :** A set’s members are immutable. So a set cannot contain another set as an element. However frozensets are immutable sets. So a set can contain a frozenset as an element.

Sets

Various operators and methods can be used to compare sets. The following sets contain the same values, so == returns `True` and != returns `False`.

```
In [1]: {1, 3, 5} == {3, 5, 1}
Out[1]: True

In [2]: {1, 3, 5} != {3, 5, 1}
Out[2]: False
```

The < operator tests whether the set to its left is a **proper subset** of the one to its right—that is, all the elements in the left operand are in the right operand, and the sets are not equal:

```
In [3]: {1, 3, 5} < {3, 5, 1}
Out[3]: False

In [4]: {1, 3, 5} < {7, 3, 5, 1}
Out[4]: True
```

The <= operator tests whether the set to its left is an **improper subset** of the one to its right—that is, all the elements in the left operand are in the right operand, and the sets might be equal:

Sets

```
In [5]: {1, 3, 5} <= {3, 5, 1}
Out[5]: True

In [6]: {1, 3} <= {3, 5, 1}
Out[6]: True
```

You may also check for an improper subset with the set method **issubset**:

```
In [7]: {1, 3, 5}.issubset({3, 5, 1})
Out[7]: True

In [8]: {1, 2}.issubset({3, 5, 1})
Out[8]: False
```

The > operator tests whether the set to its left is a **proper superset** of the one to its right—that is, all the elements in the right operand are in the left operand, and the left operand has more elements:

```
In [9]: {1, 3, 5} > {3, 5, 1}
Out[9]: False

In [10]: {1, 3, 5, 7} > {3, 5, 1}
Out[10]: True
```

Sets

The >= operator tests whether the set to its left is an **improper superset** of the one to its right—that is, all the elements in the right operand are in the left operand, and the sets might be equal:

```
In [11]: {1, 3, 5} >= {3, 5, 1}
Out[11]: True

In [12]: {1, 3, 5} >= {3, 1}
Out[12]: True

In [13]: {1, 3} >= {3, 1, 7}
Out[13]: False
```

You may also check for an improper superset with the set method **`issuperset`**:

```
In [14]: {1, 3, 5}.issuperset({3, 5, 1})
Out[14]: True

In [15]: {1, 3, 5}.issuperset({3, 2})
Out[15]: False
```

The argument to `issubset` or `issuperset` can be *any* iterable. When either of these methods receives a non-set iterable argument, it first converts the iterable to a set, then performs the operation.

Sets

- 4 set operations will be discussed
  - Union : |
  - Intersection : &
  - Set Difference : -
  - Symmetric Difference : ^
  - Disjoint

Union

The **union** of two sets is a set consisting of all the unique elements from both sets. You can calculate the union with the **| operator** or with the set type's **union** method:

```
In [1]: {1, 3, 5} | {2, 3, 4}
Out[1]: {1, 2, 3, 4, 5}

In [2]: {1, 3, 5}.union([20, 20, 3, 40, 40])
Out[2]: {1, 3, 5, 20, 40}
```

The operands of the binary set operators, like `|`, must both be sets. The corresponding set methods may receive any iterable object as an argument—we passed a list. When a mathematical set method receives a non-set iterable argument, it first converts the iterable to a set, then applies the mathematical operation. Again, though the new sets' string representations show the values in ascending order, you should not write code that depends on this.

Intersection

The **intersection** of two sets is a set consisting of all the unique elements that the two sets have in common. You can calculate the intersection with the **& operator** or with the set type's **intersection** method:

```
In [3]: {1, 3, 5} & {2, 3, 4}
Out[3]: {3}

In [4]: {1, 3, 5}.intersection([1, 2, 2, 3, 3, 4, 4])
Out[4]: {1, 3}
```

Sets

Difference

The **difference** between two sets is a set consisting of the elements in the left operand that are not in the right operand. You can calculate the difference with the **- operator** or with the set type's **`difference`** method:

```
In [5]: {1, 3, 5} - {2, 3, 4}
Out[5]: {1, 5}

In [6]: {1, 3, 5, 7}.difference([2, 2, 3, 3, 4, 4])
Out[6]: {1, 5, 7}
```

Symmetric Difference

The **symmetric difference** between two sets is a set consisting of the elements of both sets that are not in common with one another. You can calculate the symmetric difference with the **^ operator** or with the set type's **`symmetric_difference`** method:

```
In [7]: {1, 3, 5} ^ {2, 3, 4}
Out[7]: {1, 2, 4, 5}

In [8]: {1, 3, 5, 7}.symmetric_difference([2, 2, 3, 3, 4, 4])
Out[8]: {1, 2, 4, 5, 7}
```

Sets

- The operators |, &, -, ^ can be converted into their augmented assignment form.
- s1 = s1 | {3,4,5} can be reduced to
- s1 |= {3,4,5}.
- Not available for disjoint operations
- Another way is to use
  - intersection_update()
  - difference_update()
  - symmetric_difference_update()

```
set1 = {1,2,3}

set1 |= {4}
print(set1)

set1 &= {1,2}
print(set1)

set1 ^= {3,4,5}
print(set1)

set1 -= {3,4,5}
print(set1)
```

✓ 0.0s

```
{1, 2, 3, 4}
{1, 2}
{1, 2, 3, 4, 5}
{1, 2}
```

```
set1 = {1,2,3}
set1.update({4})
print(set1)
set1.intersection_update({1,2})
print(set1)
set1.symmetric_difference_update({3,4,5})
print(set1)
set1.difference_update({3,4,5})
print(set1)
```

✓ 0.0s

```
{1, 2, 3, 4}
{1, 2}
{1, 2, 3, 4, 5}
{1, 2}
```

Set Comprehension

Like dictionary comprehensions, you define set comprehensions in curly braces. Let's create a new set containing only the unique even values in the list `numbers`:

```
In [1]: numbers = [1, 2, 2, 3, 4, 5, 6, 6, 7, 8, 9, 10, 10]

In [2]: evens = {item for item in numbers if item % 2 == 0}

In [3]: evens
Out[3]: {2, 4, 6, 8, 10}
```